Implémentation de LeNet-5 en NumPy pour la reconnaissance de caractères amazighes manuscrits (base AMHCD, images RGB)

Bilal El Ibrahimi — Master IAA, FPO, 2024-2025 — Encadrement : Pr. M. Benaddy, Pr. A. Hadri

Code (notebook) : [lien GitHub à insérer] — Base : Kaggle, *Amazigh Handwritten Character Database (AMHCD)*

Résumé

Cet article présente une réimplémentation complète de LeNet-5 en NumPy (couches, rétropropagation et optimiseurs écrits à la main, sans framework) pour classer les 33 caractères Tifinagh de la base AMHCD en images couleur 32×32×3. Le réseau compte 63 961 paramètres. Les gradients sont validés par différences finies (erreur relative < 1e-7). Sur le jeu de test, le modèle atteint **[ACC_TEST] %** d'accuracy et un F1 macro de **[F1]**. Nous comparons trois optimiseurs (SGD, SGD avec moment, Adam, RMSprop), deux activations (tanh, ReLU) et deux types de pooling (moyen, max).

Mots-clés : LeNet-5, CNN, Tifinagh, AMHCD, rétropropagation, Adam.

1. Introduction

La reconnaissance de caractères manuscrits amazighes est un problème de classification d'images à 33 classes, avec une forte variabilité d'écriture. LeNet-5 (LeCun et al., 1998) est l'un des premiers réseaux convolutifs efficaces ; sa petite taille en fait un bon support pédagogique pour comprendre chaque étape : convolution, sous-échantillonnage, couches denses, rétropropagation.

Dans le TP précédent, LeNet-5 était appliqué à des images en niveaux de gris. Ici, nous le reprogrammons pour la base **AMHCD en RGB** : l'entrée passe à 3 canaux, ce qui modifie la première convolution (456 paramètres au lieu de 156). Les objectifs sont : (i) implémenter toutes les couches comme fonctions paramétrables, (ii) entraîner et évaluer le réseau (train / validation / test), (iii) analyser les résultats quantitativement et qualitativement, (iv) comparer des variantes (bonus).

2. Matériel et méthodes

2.1 Données

La base AMHCD contient des images manuscrites des 33 lettres Tifinagh, organisées en un dossier par classe. Chaque image est lue en RGB, redimensionnée en 32×32 (interpolation bilinéaire) et mise à l'échelle dans [0, 1]. Le découpage est **stratifié 70 % / 15 % / 15 %** (train / validation / test, graine 42) : [N_TRAIN] / [N_VAL] / [N_TEST] images. Chaque canal est standardisé avec la moyenne et l'écart-type calculés **sur le train uniquement**, afin d'éviter toute fuite d'information vers la validation et le test.

2.2 Architecture

Couche	Opération	Sortie	Paramètres
Entrée	image RGB	3@32×32	0
C1	convolution 5×5, 6 filtres, tanh	6@28×28	456
S2	pooling moyen 2×2, stride 2	6@14×14	0
C3	convolution 5×5, 16 filtres, tanh	16@10×10	2 416
S4	pooling moyen 2×2, stride 2	16@5×5	0
C5	dense 400→120, tanh	120	48 120
F6	dense 120→84, tanh	84	10 164
Sortie	dense 84→33, softmax	33	2 805

Total : **63 961 paramètres**. Les nombres de filtres, tailles de noyau, tailles des couches denses, activation, type de pooling, padding et schéma d'initialisation (Xavier, He, LeCun) sont des paramètres de la fonction `build_lenet5`. La couche C5 est implémentée comme une couche dense appliquée au vecteur aplati 16×5×5 = 400, ce qui équivaut à une convolution 5×5 sur des cartes 5×5.

2.3 Formules mathématiques

Convolution (stride 1, padding p) pour une entrée x de taille N×C×H×W et des filtres W de taille F×C×k×k :

- y[n,f,i,j] = b[f] + Σ_c Σ_a Σ_b W[f,c,a,b] · x[n,c,i+a,j+b]

- taille de sortie : $H_{out} = H + 2p - k + 1$ (32 → 28 pour C1, 14 → 10 pour C3).

Pooling moyen k×k, stride k : $y[n,c,i,j] = (1/k^2) \Sigma_{a,b} x[n,c,ki+a,kj+b]$. Pour le pooling max, y est le maximum de la fenêtre.

Couche dense : $y = xW + b$.

Activations : tanh(z), avec $\tanh'(z) = 1 - \tanh^2(z)$; ReLU(z) = max(0, z), avec ReLU′(z) = 1 si z > 0 et 0 sinon.

Softmax et entropie croisée : $p[n,k] = \exp(z[n,k]) / \Sigma_j \exp(z[n,j])$ et $L = -(1/N) \Sigma_n \log p[n, y_n]$. Le gradient par rapport aux logits est (p − onehot(y)) / N. Le softmax est calculé de façon stable en soustrayant le maximum des logits.

Rétropropagation. Avec $\delta = \partial L/\partial y$ la sortie amont d'une couche :

- Convolution : $\partial L/\partial W[f,c,a,b] = \Sigma_{n,i,j} \delta[n,f,i,j] \cdot x[n,c,i+a,j+b]$; $\partial L/\partial b[f] = \Sigma_{n,i,j} \delta[n,f,i,j]$; $\partial L/\partial x[n,c,i,j] = \Sigma_{f,a,b} \delta[n,f,i-a,j-b] \cdot W[f,c,a,b]$ (convolution « pleine » de δ avec le filtre retourné).
- Pooling moyen : $\partial L/\partial x = \delta / k^2$ réparti sur toute la fenêtre. Pooling max : δ est transmis à l'élément maximal.
- Dense : $\partial L/\partial W = x^T\delta$; $\partial L/\partial b = \Sigma_n \delta$; $\partial L/\partial x = \delta W^T$.

Optimiseurs (η : pas d'apprentissage, g : gradient, λ : décroissance des poids L2 ajoutée au gradient) :

- SGD : $\theta \leftarrow \theta - \eta g$. Avec moment μ : $v \leftarrow \mu v + g$, $\theta \leftarrow \theta - \eta v$.
- RMSprop : $s \leftarrow \rho s + (1-\rho)g^2$, $\theta \leftarrow \theta - \eta g / (\sqrt{s} + \varepsilon)$.
- Adam : $m \leftarrow \beta_1 m + (1-\beta_1)g$; $v \leftarrow \beta_2 v + (1-\beta_2)g^2$; $\hat{m} = m/(1-\beta_1^t)$; $\hat{v} = v/(1-\beta_2^t)$; $\theta \leftarrow \theta - \eta \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$, avec $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\varepsilon = 1e\text{-}8$.

2.4 Protocole d'entraînement et d'évaluation

Mini-lots de 64 images mélangés à chaque époque, perte d'entropie croisée, [EPOCHS] époques, initialisation de Xavier (graine 42). Configuration de référence : Adam, η = 1e-3 avec décroissance de 3 % par époque, λ = 1e-4. À chaque époque, la validation est évaluée ; le **meilleur modèle selon l'accuracy de validation** est conservé. Le jeu de test n'est utilisé qu'une fois, pour l'évaluation finale. Métriques : accuracy, F1 macro, matrice de confusion, précision et rappel par classe.

2.5 Validation de l'implémentation

Les gradients analytiques ont été comparés aux différences finies centrées, $(L(\theta+\varepsilon) - L(\theta-\varepsilon)) / 2\varepsilon$ avec ε = 1e-5, en float64, pour les activations tanh, ReLU et sigmoïde, les deux types de pooling et une convolution avec padding. L'erreur relative maximale observée est de l'ordre de 1e-8 (au plus 3e-8), ce qui valide les rétropropagations.

3. Résultats

> **À compléter après l'exécution du notebook sur la vraie base AMHCD.** Les valeurs entre crochets sont à remplacer par celles affichées par le notebook (fichier `results.json`). Les figures sont enregistrées dans le dossier `figures/`.

3.1 Résultats quantitatifs

Ensemble	Perte	Accuracy
Train	[ ]	[ ]
Validation	[ ]	[ ]
Test	[ ]	[ ] (F1 macro : [ ])

Meilleure époque : [ ]. Temps d'entraînement : [ ] s.

Figure 1 — Courbes de loss et d'accuracy (train / validation) : `figures/courbes_loss_acc.png`. Commentaire à rédiger : vitesse de convergence, écart train/validation (sur-apprentissage éventuel).

Figure 2 — Matrice de confusion normalisée sur le test : `figures/matrice_confusion.png`. Commentaire : classes les moins bien reconnues [ ] et paires les plus confondues [ ] (le notebook les affiche).

3.2 Résultats qualitatifs

Figure 3 — Cartes caractéristiques de C1, S2, C3 et S4 pour une image de test : `figures/cartes_caracteristiques.png`. Commentaire : C1 réagit aux contours et orientations du trait, S2 les lisse, C3 combine ces contours en motifs plus complexes.

Figure 4 — Filtres appris de C1 (5×5×3) : `figures/filtres_C1.png`.

Figure 5 — Exemples mal classés : `figures/erreurs.png`. Commentaire : lettres de forme proche, écriture ambiguë.

3.3 Comparaison de variantes (bonus)

Même graine, même nombre d'époques ([ABL_EPOCHS]) pour toutes les variantes ; sélection sur la validation.

Variante	Acc. validation	Acc. test	F1 macro	Temps (s)
Adam / tanh / pooling moyen (référence)	[ ]	[ ]	[ ]	[ ]
SGD / tanh / moyen	[ ]	[ ]	[ ]	[ ]
SGD + moment / tanh / moyen	[ ]	[ ]	[ ]	[ ]
RMSprop / tanh / moyen	[ ]	[ ]	[ ]	[ ]
Adam / ReLU / moyen	[ ]	[ ]	[ ]	[ ]
Adam / tanh / max	[ ]	[ ]	[ ]	[ ]

Figure 6 — Loss et accuracy de validation des variantes : `figures/comparaison.png`.

4. Discussion

À rédiger avec les résultats réels. Pistes d'analyse : (i) comparer la performance obtenue à celle du TP en niveaux de gris et dire si la couleur apporte de l'information utile pour des caractères manuscrits ; (ii) expliquer les différences entre optimiseurs (Adam adapte le pas par paramètre, SGD simple dépend fortement de η) ; (iii) discuter l'effet de tanh vs ReLU et du pooling moyen vs max ; (iv) analyser les confusions entre lettres graphiquement proches ; (v) relever les limites : un seul découpage et une seule graine (pas d'intervalle de confiance), redimensionnement 32×32 qui peut effacer des détails, absence d'augmentation de données.

Améliorations possibles : augmentation de données (petites rotations, translations), dropout ou normalisation par lots, validation croisée, plusieurs graines pour estimer la variance.

5. Conclusion

Nous avons réimplémenté LeNet-5 de bout en bout en NumPy, avec des gradients vérifiés numériquement, et l'avons appliqué à la classification des 33 caractères Tifinagh de la base AMHCD en RGB. Le réseau de référence atteint [ACC_TEST] % sur le test. [Phrase de conclusion sur la meilleure variante et les perspectives.]

Références

[1] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, « Gradient-Based Learning Applied to Document Recognition », *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] D. P. Kingma and J. Ba, « Adam: A Method for Stochastic Optimization », arXiv:1412.6980, 2014.

[3] Base AMHCD, Kaggle : https://www.kaggle.com/datasets/benaddym/amazigh-handwritten-character-database-amhcd